

FDO IMP FOR UT-1

-Harsh Shelke

FDO Unit 1: Modern Application Architecture & Core Components

Q1. Compare Monolithic and Microservices Architectures. Discuss their benefits and challenges. (5 Marks)

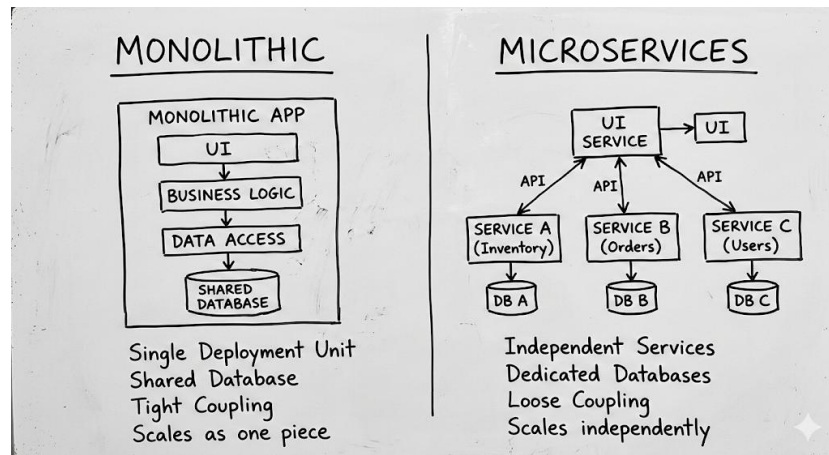
Answer:

- **Monolithic Architecture:** A single, unified application where all components (UI, business logic, database access) are tightly integrated and deployed together as a single unit.
 - **Pros:** Simple to develop initially, and easy to test and debug.
 - **Cons:** Hard to scale specific features, high risk during updates, and poor fault isolation.
- **Microservices Architecture:** Structures an application as a collection of small, independent services, each responsible for a specific business capability.
 - **Pros:** Independent scaling, faster feature delivery, and better fault isolation.
 - **Cons:** Network latency, complex debugging, and operational overhead.

Key Differences:

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent services
Scalability	Entire app	Per service
Fault Isolation	Poor	Strong
Complexity	Low initially	High operationally

Real-World Example: Applications like Netflix, Amazon, and Uber rely on microservices so that if one feature (like the recommendation engine) crashes, the rest of the application remains available.



Q2. What is Event-Driven Architecture (EDA)? Explain its core components and benefits. (5 Marks)

Answer:

Definition: An event-driven architecture (EDA) uses events to trigger and communicate between decoupled services. It moves away from traditional request-response mechanisms.

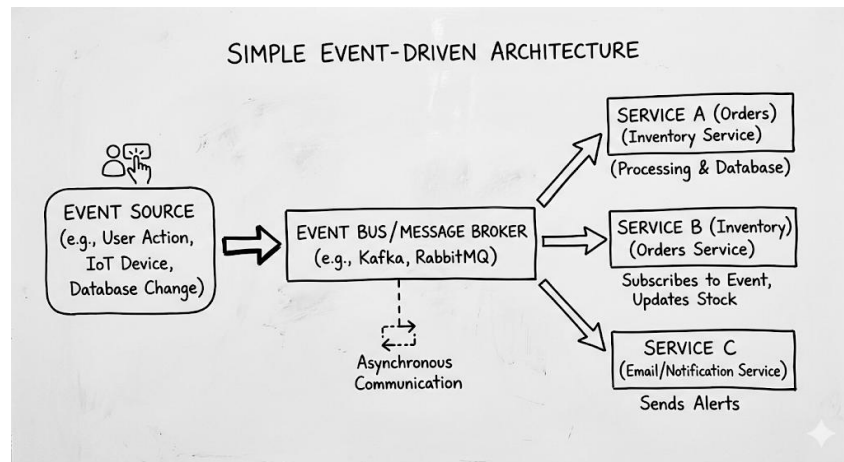
Core Components:

- **Event:** A lightweight, immutable piece of data that represents a meaningful change or occurrence within a system.
- **Event Producer:** The component (like a user interface or sensor) responsible for generating and emitting events.
- **Event Consumer:** The service or module that listens for and reacts to specific events.
- **Event Broker:** An intermediary (like Apache Kafka or RabbitMQ) that efficiently routes events from producers to consumers.

Benefits:

- **Scalability:** Systems can scale horizontally by easily adding more event consumers.
- **Loose Coupling & Flexibility:** Components are independent, allowing you to modify them without affecting existing ones.
- **Real-time Responsiveness:** Enables immediate reactions to user actions.

Real-World Example: In an E-commerce system, placing an order generates an event. The inventory, billing, and shipping services all consume this event and do their jobs simultaneously without waiting on each other.



Q3. Explain the core concepts of Container Orchestration. Why is it necessary? (5 Marks)

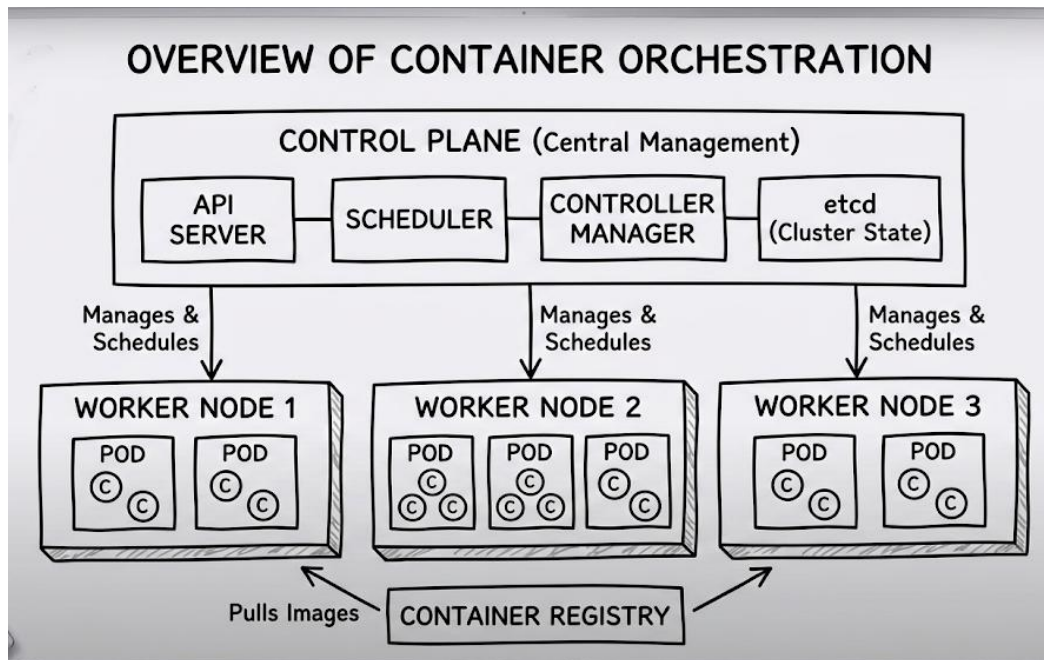
Answer:

Why it is necessary: While containers are lightweight and fast, managing hundreds or thousands of them manually in a production environment is impossible. Container orchestration automates their deployment, management, scaling, and networking.

Core Concepts (Write all 5 for full marks):

1. **Scheduling:** Automatically places containers on available worker nodes based on resources.
2. **Networking:** Enables container-to-container communication across different nodes.
3. **Service Discovery:** Allows services to find each other dynamically as containers spin up and down.
4. **Scaling:** Automatically performs horizontal scaling (adding replicas) based on current workload and load.
5. **Self-Healing:** Automatically detects failures to restart failed containers and replace unhealthy instances.

Real-World Example: Using Kubernetes to automatically spin up 50 extra web server containers during a flash sale, and automatically destroying them when the traffic drops to save costs.



Q4. Discuss the operational challenges of managing many containers and how to solve them. (5 Marks)

Answer:

Context: Shifting to microservices and containers shifts complexity from the application code to the infrastructure.

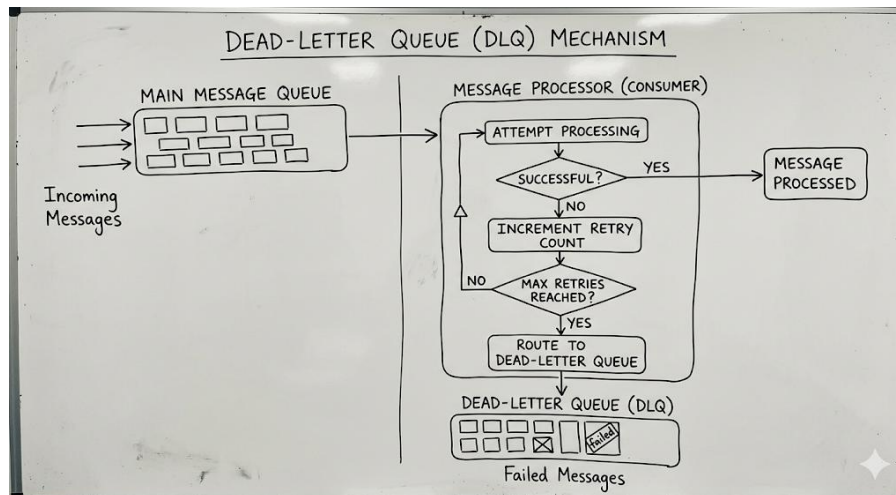
Key Challenges:

- **Monitoring & Logging:** Tracking the health and logs of thousands of containers that are constantly created and destroyed.
- **Configuration Management:** Ensuring every container receives the correct configurations seamlessly.
- **Security & Access Control:** Securing a highly distributed environment and managing access.
- **Network Complexity:** Handling the massive volume of network traffic between hundreds of microservices.

Solution Approaches:

- **Centralized Logging & Observability:** Implementing centralized logging and observability tools (like Prometheus or Grafana) to get a unified view of system health.
- **Infrastructure as Code (IaC):** Using tools like Terraform to manage and provision infrastructure automatically through code rather than manual processes.

Real-World Example: A DevOps engineer writing a Terraform script to define network rules for 1,000 containers so they don't have to be configured manually one by one.



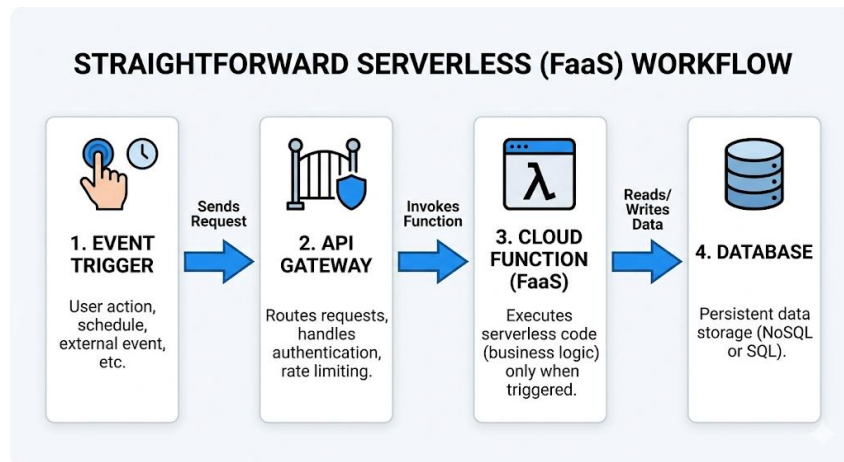
Q5. Explain the principles of Fault-Tolerant Design (Retries, Circuit Breakers, Dead-Letter Queues). (5 Marks)

Answer:

Definition: Fault tolerance is the ability of a system to continue functioning despite inevitable failures in distributed systems.

Core Principles:

- **Retries:** When a request fails, the system automatically re-attempts it. It should use exponential backoff (waiting progressively longer between tries) to avoid overwhelming a struggling service.
 - *Example:* Your phone attempting to resend a failed text message a few seconds later.
- **Circuit Breaker:** A pattern that stops requests to a failing service to prevent cascading failures. If a service fails repeatedly, the circuit "trips" open, failing fast instead of waiting for timeouts.
 - *Example:* A home's electrical circuit breaker tripping during a power surge to prevent damage.
- **Dead-Letter Queues (DLQ):** A special queue that stores messages that cannot be processed successfully after all retries. This prevents data loss and allows for manual debugging.
 - *Example:* A "dead letter office" in a postal system where mail with bad addresses is sent for manual inspection.



Q6. Detail Serverless Computing and Functions-as-a-Service (FaaS), including triggers and configurations. (5 Marks)

Answer:

Definitions:

- **Serverless Computing:** A cloud execution model where developers write code without managing servers. The core idea is "Focus on code, not infrastructure".
- **Functions-as-a-Service (FaaS):** Small, single-purpose, stateless functions that execute in response to events, using a pay-per-use billing model.

Key Components:

- **Triggers & Event Sources:** Functions are awakened by triggers such as HTTP requests, database changes, file uploads, or message queues.
- **Function Configuration:** Crucial for performance and cost control. Includes Memory allocation, Timeout limits (to prevent infinite loops), and Concurrency settings.
- **Environment Configuration:** Storing sensitive data like Database URLs and API keys as environment variables. Best practice dictates never hardcoding secrets.

Real-World Example: Deploying an AWS Lambda function that automatically executes only when a user uploads an image to an Amazon S3 bucket, resizes the image, and immediately shuts down.

Q7. Describe modern Data and Storage Patterns: NoSQL Databases, Object Storage, and Caching. (5 Marks)

Answer:

Modern applications handle massive, diverse data types and require specialized storage solutions.

1. NoSQL Databases: Non-relational databases that do not require a fixed schema and scale out horizontally.

- **Key-Value Stores:** Stores data as key-value pairs. Extremely fast and highly scalable. Examples: Redis, DynamoDB. Best for caching or session data.
- **Document Stores:** Stores data as JSON-like documents, offering a flexible schema. Examples: MongoDB, CouchDB. Best for user profiles and content management.

2. Object Storage & Data Lifecycle Management:

- **Object Storage:** Used to store unstructured data (like images and videos) accessed via APIs rather than a file system. Examples: Amazon S3, Azure Blob Storage.
- **Lifecycle Management:** Automatic cost optimization by moving data through tiers: Hot (frequently accessed) → Cold → Archive.

3. Caching Strategies:

- **Purpose:** Reduces latency and lowers the load on the primary database.
- **Patterns:** Uses in-memory caches (like Redis or Memcached) with patterns like Read-through, Write-through, and Cache-aside.

Real-World Example: Swiggy using an in-memory cache (Redis) to load nearby restaurants instantly, a Document Database (MongoDB) to store complex restaurant menus, and Object Storage (Amazon S3) to host the food images.

FDO Unit 2: APIs, Communication & Programmatic Interaction

Q1. Explain the core principles of REST APIs. What is an API Gateway, and why is it critical in modern architecture? (5 Marks)

Answer:

REST API Principles: REST (Representational State Transfer) is an architectural style for designing networked applications. Its core principles include:

- **Stateless Communication:** The server does not store any client context between requests; every request must contain all needed information.
- **Resource-based URLs:** Data and services are treated as resources identified by specific URLs.
- **Standard HTTP Methods:** Uses standard actions like GET (Read), POST (Create), PUT/PATCH (Update), and DELETE (Remove).

API Gateway: An API Gateway acts as a single entry point or "front door" for all client requests before they reach individual backend microservices.

Key Responsibilities / Why it is critical:

- **Routing & Transformation:** Directs requests to the correct service and modifies the request/response if needed.
- **Rate Limiting & Throttling:** Prevents servers from being overwhelmed by too many requests.
- **Authentication & Validation:** Checks if the user is authorized before letting them access the backend.

Real-World Example: When you open the Netflix app, your phone talks to a single API Gateway. The Gateway then routes your login details to the "Auth Service" and your movie request to the "Video Streaming Service."

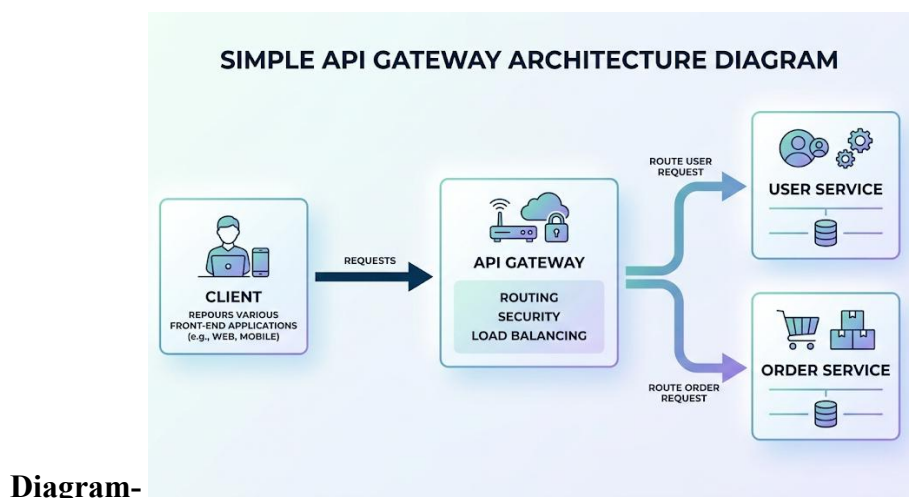


Diagram-

Q2. Differentiate between Synchronous and Asynchronous communication models. Which is better for cloud scalability? (5 Marks)

Answer:

Synchronous Communication: A request-response pattern where the client sends a request and is blocked (must wait) until the service responds.

- **Pros:** Simple logic, immediate feedback.
- **Cons:** Tightly coupled, limited scalability, and poor fault tolerance (if the service is down, the client fails).
- **Examples:** REST API calls, gRPC calls.

Asynchronous Communication: The producer sends a message to a queue/topic and does not wait for it to be processed. The consumer processes the message independently later.

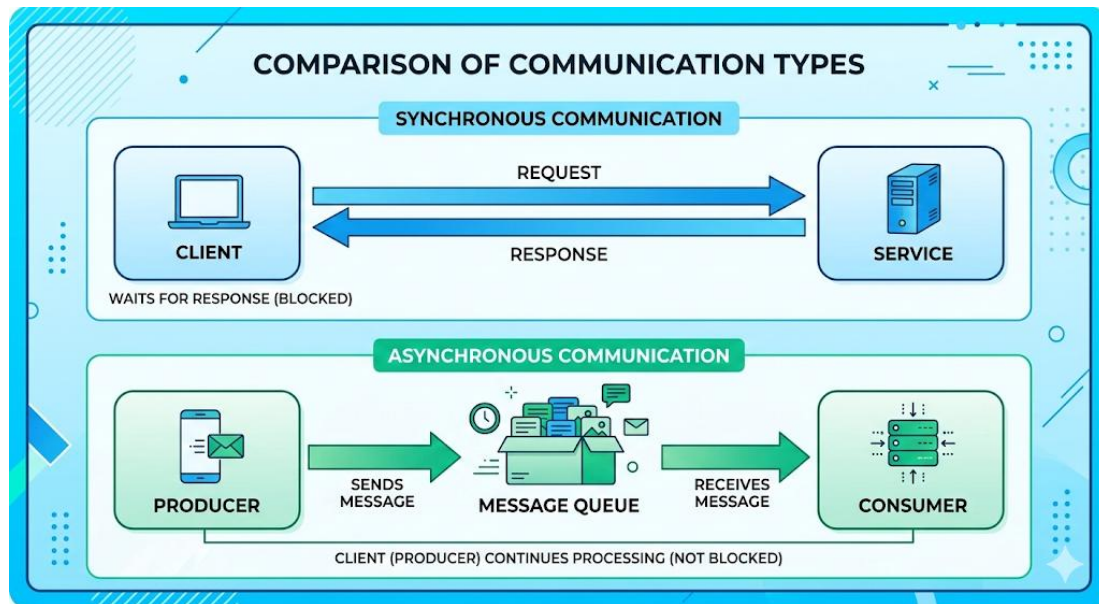
- **Pros:** Loosely coupled, high scalability, and high fault tolerance (system remains responsive even if consumers are slow).
- **Cons:** Complex to debug, eventual consistency.
- **Examples:** Message Queues, Event Streams.

Comparison Table:

Aspect	Synchronous	Asynchronous
Waiting	Client waits	No waiting
Coupling	Tight	Loose
Scalability	Limited	High
Fault Tolerance	Low	High
Example	REST API call	Message Queue

Conclusion: Asynchronous communication is better for cloud scalability because it allows systems to continue working without being blocked by slow downstream services.

Real-World Example: Payment processing. Verifying your card details is *Synchronous* (you must wait to see if it's approved). Sending your PDF receipt via email is *Asynchronous* (the system queues it and sends it in the background while you continue browsing).



Q3. Explain the concepts of Message Queues and Message Topics in messaging systems. Provide relevant use cases. (5 Marks)

Answer:

Messaging systems act as the backbone for decoupling services in cloud-native architectures. There are two main models:

1. Message Queues (Point-to-Point):

- **Concept:** A one-to-one communication model. One producer sends a message, and it is processed by exactly *one* consumer. Once processed, the message is removed.
- **Use Cases:** Order processing, background jobs, task scheduling.
- **Advantage:** Queues gracefully absorb massive traffic spikes without crashing the backend workers.

2. Message Topics (Publish-Subscribe):

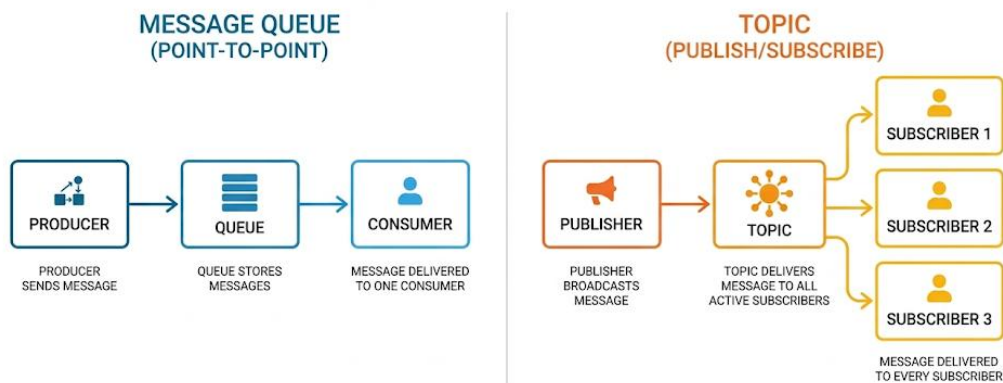
- **Concept:** A one-to-many communication model. One producer (publisher) sends a message to a topic, and *multiple* consumers (subscribers) all receive a copy of that message.
- **Use Cases:** System-wide notifications, fan-out workloads, and broadcasting updates.

Real-World Example:

- **Queue:** Uploading a video to YouTube. Your video goes into a queue to be compressed. Only *one* server needs to process your specific video.
- **Topic:** When a YouTuber uploads a video, the system publishes a message to a "New Upload" topic. Millions of subscribers (consumers) receive a push notification simultaneously.

Diagram –

MESSAGING PATTERNS: QUEUES vs TOPICS



Q4. Compare Software Development Kits (SDKs) and Command Line Interfaces (CLIs). Explain their role in cloud computing. (5 Marks)

Answer:

Both SDKs and CLIs are tools used to programmatically interact with cloud services without using the web console, allowing for automation at scale.

Software Development Kits (SDKs):

- Language-specific libraries (e.g., Python, Java, Node.js) that developers embed directly into their application code.
- They abstract low-level API calls, making cloud interaction faster to code.
- **Role:** Used for runtime integration (e.g., your Python backend app saving a user's profile picture directly to AWS S3).

Command Line Interfaces (CLIs):

- Terminal-based tools that allow users to run commands to manage cloud resources.
- They are language-agnostic and highly scriptable.
- **Role:** Used primarily in IT operations, DevOps automation, and CI/CD pipelines (e.g., a bash script that automatically creates 10 virtual machines).

Key Differences:

Feature	SDKs	CLIs
Primary Use	Application code	Automation & IT Ops
Language	Language dependent (Python, Java)	Language agnostic (Bash, Shell)
Integration	Runtime integration	Pipeline / CI/CD integration

Q5. Explain how Event Streams and Event Buses help in designing decoupled service communication flows. (5 Marks)

Answer:

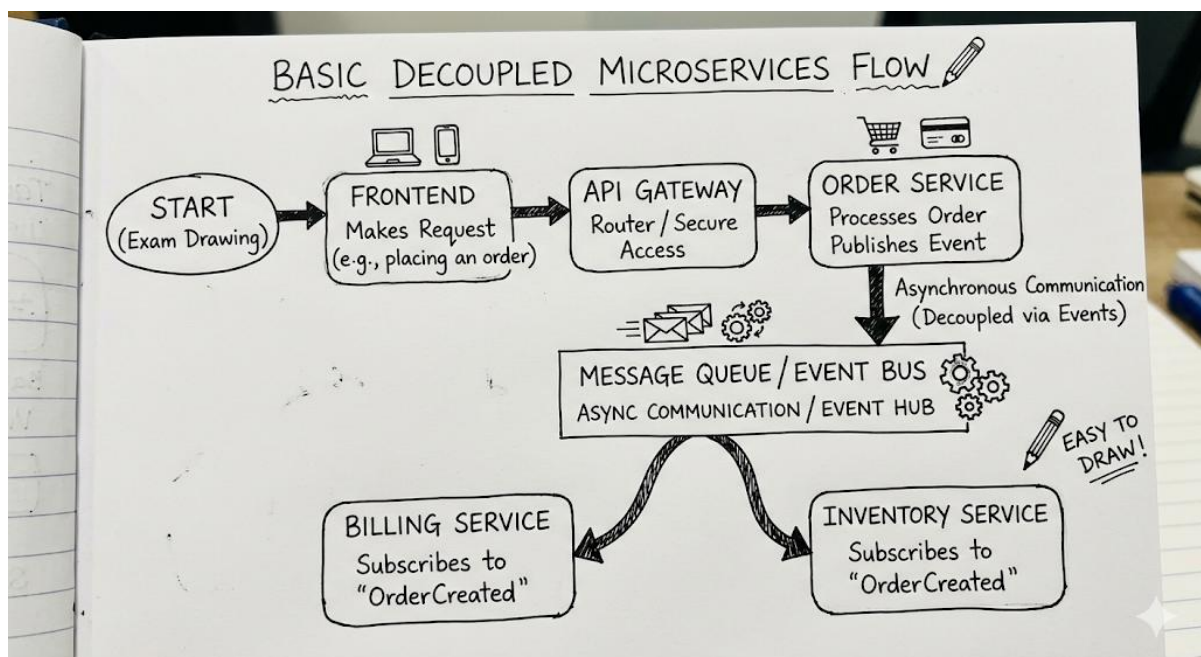
In modern DevOps, designing loosely coupled systems is best practice. This avoids direct service dependencies and ensures systems can fail gracefully.

- **Event-Driven Architecture (EDA):** A design where systems react to "events" (state changes) rather than direct requests. Producers generate events without needing to know who the consumers are.
- **Event Streams:** Provide a continuous flow of ordered, replayable events. They handle massive, high-throughput data (like website clickstreams or IoT sensor data).
- **Event Buses:** Act as a central event routing layer. They take incoming events and use rules to decide exactly which services should receive them.

Designing the Flow: A highly decoupled flow starts at the Frontend, hits the API Gateway, and instead of that service calling other services directly, it drops a message into a Message Queue or Event Bus for downstream processing.

Real-World Example: In a smart home, a motion sensor (producer) sends an event to the Event Bus. The bus routes this event to the "Lighting Service" (to turn on lights) and the "Security Service" (to log the motion). The sensor knows nothing about the lights or cameras; they are completely decoupled.

Diagram-



Q6. How do APIs, Messaging Systems, and Event-Driven architectures combine to form a "Unified Cloud Architecture"? Explain the flow of a decoupled microservice system. (5 Marks)

Answer:

A Unified Cloud Architecture combines synchronous APIs and asynchronous messaging to build systems that scale effortlessly, fail gracefully, and evolve continuously.

The Decoupled Flow (Step-by-Step):

1. **Client Request:** A user interacts with the Frontend/Client, which sends a synchronous HTTP request.
2. **The Front Door:** The request hits the API Gateway, which handles authentication, rate limiting, and routes the request securely to the correct internal service.
3. **Synchronous Processing:** The target Microservice (e.g., the Order Service) receives the request and processes the immediate business logic.
4. **Asynchronous Handoff (The Decoupling):** Instead of waiting for other services to finish related tasks (like sending a confirmation email or updating inventory), the Order Service simply publishes an event to a Message Queue, Topic, or Event Bus.
5. **Background Execution:** Downstream Cloud Services and other microservices act as consumers, pulling those messages from the queue/topic and processing them asynchronously in the background.
6. **The DevOps Mindset:** In this architecture, APIs are treated as products, communication is an intentional design decision, events act as signals, and automation (using SDKs and CLIs) is non-negotiable.